

MULTI-AGENT WORLD: 面向严格成功率-关键步数 PARETO 最优的自适应任务编排

Anonymous authors

Paper under double-blind review

ABSTRACT

大语言模型驱动的 Agent 已能够在可执行环境中完成多轮工具调用,但在包含多个子目标、动态环境状态和跨阶段约束的长程任务中,单 Agent 需在不断增长的同一上下文中维护全局计划、中间证据与失败历史,容易出现上下文过载、信息遗失和错误累积;串行执行所有动作还使任务完成时间随执行步数增长。Multi-Agent 可以通过上下文分片、并行执行和独立核验缓解这些问题,却同时引入是否委派、如何拆分、哪些分支并行、何时同步以及如何恢复失败等任务编排问题。为此,我们提出 MULTI-AGENT WORLD,一个面向模型任务编排能力增强的三阶段训练框架:在环境构建阶段,从描述有状态应用 MCP 规范出发,构建可交互、可重置环境及基础工具调用任务;在编排数据合成阶段,依据基础任务的读写集合、前后置条件与验证约束,组合出覆盖串行、并行、混合、竞争和自适应恢复结构的复杂任务;在模型训练阶段,先训练可靠的工具型 Worker,再以其作为 Sub-Agent 训练 Orchestrator。除严格终局验证奖励外,我们设计执行步骤感知的过程奖励,对中间子任务与最终结果共同打分,并使用 Critical Tool Steps (CTS) 与总工作量约束执行成本,从而联合优化编排正确性和执行效率。评测在相同模型、工具、硬件和资源预算下,统一比较 Single-Agent ReAct、串行 Multi-Agent、激进全并行 Multi-Agent 与学习型自适应编排,并联合报告严格任务成功率、真实完成时间、作为逻辑延迟代理的 CTS 与总工作量。**[待补:结果:主实验完成后补入最重要的定量结果及其对应的准确率-延迟结论。]**

1 引言

由大语言模型驱动的 Agent 正在从生成静态回答,走向在文件系统、数据库、网页与软件服务中持续行动。ReAct、Toolformer 与 ToolLLM 等工作表明,语言模型可以通过交替推理与工具调用完成多步任务 (Yao et al., 2023; Schick et al., 2023; Qin et al., 2024);近期的可执行环境与 Agentic RL 研究又进一步将训练对象扩展到带状态转移、可重置沙箱和程序化验证器的交互轨迹 (Wang et al., 2026; Dong et al., 2026; Cheng et al., 2026)。这些进展显著增强了模型作为执行器 (Executor) 的能力:给定当前观察,模型能够选择工具、填写参数,并根据反馈修正后续动作。

随着 Agent 从相对独立的多轮工具调用走向跨系统、长时间运行的长程任务,单 Agent 的序列执行范式开始暴露出新的瓶颈。长程任务往往包含多个子目标、持续变化的环境状态和跨阶段约束;单个模型必须在同一上下文中同时维护全局计划、中间证据、工具反馈与失败历史。随着轨迹增长,关键条件容易被后续观察淹没,造成上下文过载、目标漂移和历史信息遗失,局部错误也会沿后续步骤持续累积。与此同时,单 Agent 需要在每一轮基于不断增长的完整历史重新推理,并按顺序执行所有动作,即使其中存在相互独立的分支也无法重叠,因而任务完成时间通常随总执行步数持续增长。

Multi-Agent 为缓解这些限制提供了一条自然路径。Orchestrator 可以将不同子目标分配给具有独立上下文的 Worker,使每个 Worker 只保留完成局部任务所需的信息,从而降低单一上下文的状态负担;相互独立的分支可以并发执行,缩短整体关键路径;角色分工和独立核验还可以减轻规划、执行与检查由同一模型承担所造成的能力耦合。Kimi K2.5 的 Agent Swarm 与 Claude Research 的动态 Orchestrator-Worker 系统展示了这种分解和并行执行的潜力 (Kimi Team, 2026; Hadfield et al., 2025)。

然而，从单一 Agent 扩展到多 Agent 的系统并不会自动得到更好的系统。过度拆分会增加通信与交接成本，错误并行可能违反尚未识别的前置依赖，并发写入可能破坏共享状态，而重复劳动、不完整返回和冲突结果又要求主 Agent 进行同步、聚合、验证与局部重试。串行委派虽然隔离了上下文，却无法获得并行收益；无条件最大化并发度虽然可能降低理想执行时间，却会牺牲可靠性和总工作量。因此，真正缺失的不是“调用更多 Agent”的能力，而是根据任务结构决定是否委派、如何拆分、哪些分支并行、何时同步以及如何恢复失败的能力。我们将其称为任务编排（*task orchestration*）能力。

现有的环境扩展工作主要回答 Agent “在哪里学习”。Agent World Model 从场景、任务、数据库、接口与验证器合成代码驱动环境 (Wang et al., 2026)；Agent-World 将真实工具环境的扩展与跨训练轮次的自演化闭环结合 (Dong et al., 2026)；Terminal-World 用 Agent skill 统一派生终端任务、环境与教师轨迹 (Cheng et al., 2026)。这些工作为可执行、可验证的 Agent 学习提供了重要基础，但其核心学习对象仍是单一 Agent 完成部分工具调用轮次的能力。即便数据合成阶段使用 skill graph 或多个 Agent，其任务目标简单难以激发出模型编排任务的能力。因此，如何规模化的合成出可以训练提升模型编排能力的环境与问题，成为当前 Multi-Agent System 主要的难点。

为解决这个问题，我们提出 MULTI-AGENT WORLD，一个面向模型任务编排能力增强的完整训练框架。整个框架包含三个阶段。首先，在**环境构建**阶段，我们从大量的 mcp 环境中筛选出包含真实状态的 mcp 环境描述，并且在此基础上合成带有状态可交互的环境，以及基础的工具调用问题。其次，在**编排数据合成**阶段，我们依据基础任务的读写集合、前后置条件与验证约束，将其组合为串行、并行、混合、竞争和自适应恢复等复杂编排任务。最后，在**模型训练**阶段，我们先训练可靠的工具型 Worker，再使用 Worker 作为 Sub-Agent 在复杂编排任务上训练 Orchestrator。除最终任务验证奖励外，我们还设计了执行步骤感知的过程奖励：训练轨迹由最终结果和中间各个子任务共同打分；同时利用 Critical Tool Steps (CTS) 与总工作量约束执行成本，从而共同激励编排质量与执行效率。

评测方面，我们不仅统计不同方法的严格任务成功率，还在相同模型、工具、硬件和资源预算下记录真实完成时间，并采用 CTS 作为可复现的逻辑延迟代理。具体而言，我们将训练后的编排模型与 Single-Agent ReAct、串行 Multi-Agent、激进全并行 Multi-Agent 不同编排框架对比得到在准确率与延迟方面我们的方法同时取得了 SOTA。与其他工具调用 Agent 相比，MULTI-AGENT WORLD 在 BFCL, tau2-bench 等基础工具调用 benchmark 及 deepplanning, wide search, Toolathlon 等长程编排 benchmark 上平均取得了 SOTA，同时在执行时间上大幅领先[待补：结果：填入实验 accuracy 和时间 cost]。更进一步的，我们还比较了使用不同规模的主/副 Agent 在不同任务表现，探索当前这套编排框架与模型的能力上限，结果发现当前 MAW 的编排能力已经远超闭源模型。综合来看，这些结果证明 MULTI-AGENT WORLD 通过合理的编排分配可并行任务真正解决了单 Agent 在长程任务中的上下文与串行执行瓶颈。

本文的主要贡献概括如下：

- **面向编排学习的可验证环境与任务合成**。我们提出一套从有状态 MCP 描述构建可执行、可重置环境的流水线，并依据基础任务的读写集合、前后置条件和验证约束，规模化合成覆盖串行、并行、混合、竞争与失败恢复结构的复杂任务，使运行时执行拓扑选择成为可训练、可验证的问题。
- **执行步骤感知的两阶段编排训练**。我们先训练可靠的工具型 Worker，再以其作为 Sub-Agent 训练 Orchestrator；在严格终局验证奖励之外，引入由中间子任务反馈构成的过程奖励，并通过 CTS 与总工作量约束执行成本，从而联合优化编排正确性与执行效率。
- **准确率-延迟联合评测**。我们在统一模型、Worker、工具与资源预算下，系统比较 Single-Agent ReAct、串行 Multi-Agent、激进全并行 Multi-Agent 和学习型自适应编排，同时报告严格任务成功率、CTS、真实完成时间与总工作量，以隔离并量化任务编排带来的性能收益和额外成本。

2 相关工作

[待补：之后再扩充调研其他论文]

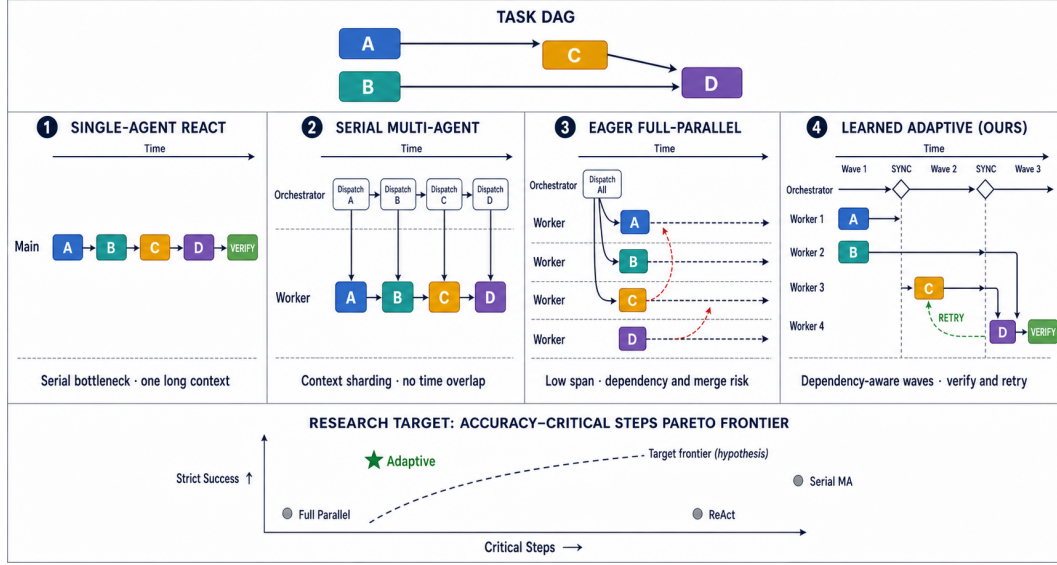


Figure 1: 四类执行策略及本文研究目标。示意任务中 A 与 B 可并行, C 依赖 A , D 依赖 B 与 C 。底部曲线表示待检验的研究假设, 而非已有实验结果。

2.1 工具型 AGENT 与 AGENTIC RL

工具增强语言模型将语言模型从静态生成器扩展为能够调用外部接口的执行器。Toolformer 研究如何通过自监督信号学习何时调用工具, ReAct 将语言推理、动作与环境观察交替组织为多步轨迹, ToolLLM 则通过大规模真实 API 数据与检索机制提升工具选择和参数生成能力 (Schick et al., 2023; Yao et al., 2023; Qin et al., 2024)。在此基础上, Agentic RL 进一步利用可执行环境的状态转移、工具反馈和终局验证信号优化多轮决策 (Wang et al., 2026; Dong et al., 2026; Cheng et al., 2026)。这条研究路线主要增强给定当前历史后选择下一项工具动作并完成任务的能力。

本文继承上述 Executor 能力, 但改变了待学习的决策层级: 策略不仅要决定下一步执行什么, 还要在测试时无法观察真实依赖图的条件下, 决定是否拆分任务、分支应串行还是并行、何时同步, 以及失败后应验证、重试还是重新规划。为隔离这部分收益, 我们固定 Worker checkpoint, 将性能差异主要归因于 Orchestrator 对运行时执行拓扑的选择, 而不是底层工具调用能力的同步提升。

2.2 可执行环境与任务合成

代码驱动并由数据库或沙箱承载的环境能够提供可重置的初始状态、一致的状态转移和程序化验证, 因此适合大规模轨迹采集与在线强化学习。近期三项工作从不同的合成原语扩展了此类训练环境。Agent World Model (AWM) 从场景与用户需求出发, 依次合成任务、数据库 schema、工具接口、后端实现与验证器, 构造全合成、数据库支持的 MCP 环境, 并用其训练多轮工具调用 Agent (Wang et al., 2026)。Agent-World 从真实世界主题出发, 通过深度检索发现与主题匹配的数据库和可执行工具集, 再结合图式与程序化任务生成、可执行奖励和跨训练轮次的能力诊断, 形成环境与策略持续共同演化的闭环 (Dong et al., 2026)。Terminal-World 则以人类编写的 Agent skill 为联合描述任务目标、环境约束与执行过程的合成原语, 将单个 skill、skill team 和 skill graph 实例化为终端任务、可执行沙箱、pytest 验证器与技能引导的教师轨迹 (Cheng et al., 2026)。其中用于构建文件与环境的多 Agent 流水线发生在数据生成阶段, 不同于本文在测试阶段由待训练 Orchestrator 动态调度 Worker 的问题。

MULTI-AGENT WORLD 不以环境规模本身作为主要差异, 而是在上述可执行、可验证范式上增加编排结构。具体而言, 我们为原子任务记录读写集合、前后置条件和验证约束, 并将其组合为包含数据依赖、共享状态冲突与失败恢复分支的长程任务。生成器掌握这些结构以构造实例、计算 Oracle 并进行分桶分析, 但真实执行图不作为训练或测试策略的输入; Orchestrator 必须根据任务文本与运行时观察推断依赖, 并选择相应执行拓扑。因此, skill graph 或任务图用于离线合成任务, 与策略在未知图上在线学习拓扑是两个不同的问题。

2.3 MULTI-AGENT 系统与 ORCHESTRATOR-WORKER 架构

Multi-Agent 系统通过独立上下文、角色分工与并行执行扩展单 Agent 的有效工作范围。Claude Research 采用动态 Orchestrator-Worker 架构，将开放式研究问题分解后交由多个 Sub-Agent 并行搜索，再由 Lead Agent 汇总结果；Kimi K2.5 的 Agent Swarm 进一步展示了自导式分解、并行 Worker 和集中聚合在长程任务中的潜力 (Hadfield et al., 2025; Kimi Team, 2026)。这些系统说明动态委派可以缓解上下文负担并缩短可并行分支的等待时间，同时也揭示了通信成本、重复工作以及强依赖任务难以并行等系统权衡。

与以端到端系统能力为主要对象的已有工作互补，本文将编排本身隔离为可训练、可验证的受约束决策问题。所有方法共享相同的 Worker、工具、初始状态与预算；任务在测试时包含未知的数据依赖和共享可变状态，错误并行会触发可复现的前置条件失败、版本冲突或终局验证失败。因而，Orchestrator 不能默认采用固定 workflow、全串行委派或最大并行，而需要针对每个任务在线选择执行拓扑。该设置也使“增加 Agent 数量”和“提升编排策略”能够被分别检验。

3 问题定义

3.1 有状态工具执行环境与任务

给定用户任务 x 与可重置的 MCP 工具环境 E ，环境表示为

$$E = (\mathcal{S}, \mathcal{A}_{\text{tool}}, \mathcal{O}, T, \rho_0), \quad (1)$$

其中 $\mathcal{S}$ 、 $\mathcal{O}$ 和 $\mathcal{A}_{\text{tool}}$ 分别是状态、观察与工具动作空间， T 是由可执行工具实现的状态转移函数， ρ_0 是 episode 初始状态分布。任务 x 包含自然语言目标 q_x 、目标输出 y_x^* 与严格终局验证器 V_x^{final} ；验证器同时检查最终回答和环境状态。

任务合成器掌握真实执行结构

$$G_x = (\mathcal{U}_x, \mathcal{D}_x, \mathcal{C}_x), \quad (2)$$

其中 $\mathcal{U}_x = \{u_1, \dots, u_m\}$ 是可验证子任务集合， $\mathcal{D}_x$ 是由数据传递或状态前置条件产生的依赖边， $\mathcal{C}_x$ 是不能安全并发执行的共享状态冲突边。每个 u_j 配有训练期局部验证器 $V_{x,j}^{\text{proc}}$ 。

3.2 ORCHESTRATOR-WORKER 推理过程

[待补：需要后面结合具体的 multi-agent 框架重新组织] 系统由可训练 Orchestrator π_o 和一组固定的 Worker π_w 组成，任一时刻最多并发执行 K 个 Worker。Orchestrator 根据历史

$$h_t = (q_x, \mathcal{A}_{\text{tool}}, a_{<t}, o_{<t}, b_t) \quad (3)$$

选择动作，其中 b_t 表示剩余 CTS、总工作量与并发预算。其动作空间为

$$\mathcal{A}_o = \{\text{act}, \text{create subagent}, \text{assign task}, \text{assign tasks}, \text{finish}\}. \quad (4)$$

其中 **act** 表示 Orchestrator 直接执行任务工具，**create subagent** 创建新的子代理，**assign task** 分配单个任务，**assign tasks** 并行分配多个任务，**finish** 表示提交答案并完成当前任务。

一次 rollout τ 不再是单一线性轨迹，而是由 Orchestrator 动作、Worker 工具分支、共享状态转移和同步边组成的动态执行 DAG H_τ 。本文学习的对象是 π_o 对 H_τ 的在线构造策略；冻结 π_w 用于将编排收益与底层工具执行能力解耦。

4 MULTI-AGENT WORLD

如图 2 所示，MULTI-AGENT WORLD 将编排能力的训练分为三个相互衔接的阶段：构建可执行的有状态世界与原子任务、将原子任务组合为需要拓扑决策的复杂任务，以及在这些任务上进行两阶段训练。本节介绍前两个阶段；第 5 节进一步介绍 Worker 与 Orchestrator 的训练方法。整个数据流水线掌握问题定义中的真实结构 G_x ，但策略仅观察自然语言任务、工具 schema 与执行反馈，因而不能通过读取图标注直接获得调度方案。

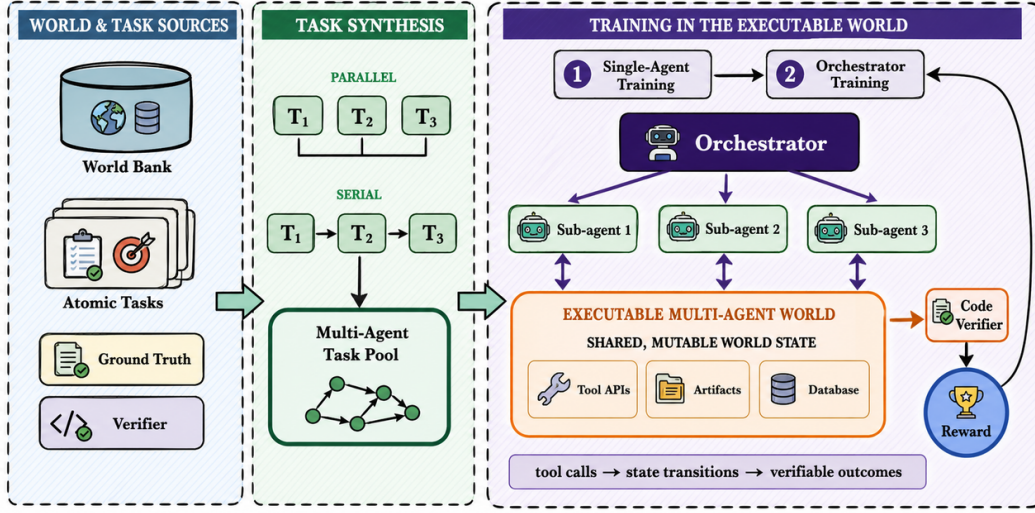


Figure 2: MULTI-AGENT WORLD 方法总览。左：从有状态 MCP 描述构建可执行、可重置的 World Bank 与原子任务。中：Serial/Parallel 是基本组合原语；递归组合以及冲突/失败注入进一步产生 Mixed、Contention 和 Recovery 任务。右：先训练工具型 Worker，再冻结 Worker 并训练 Orchestrator；图中的 Code Verifier 统指局部与终局验证器家族，为简洁起见省略 Orchestrator 直接调用任务工具的通路。真实任务图只用于数据合成和评测，不作为策略输入。

4.1 从 MCP 描述到有状态可执行世界

有状态环境筛选。 我们首先从候选 MCP 描述中筛选能够形成封闭状态转移的应用：其工具具有可解析的名称、参数和返回 schema，操作对象具有可重置的结构化状态，并且核心功能可以在不依赖登录凭证和外部在线服务的条件下离线实例化；用于共享状态编排的环境还需包含至少一项确定性写操作。纯文本检索、无持久对象以及无法构造确定性初始状态的候选被排除。该筛选使每个保留环境都能实例化为第 3 节定义的 $E = (\mathcal{S}, \mathcal{A}_{\text{tool}}, \mathcal{O}, T, \rho_0)$ ，而非仅提供一组脱离状态的 API 名称。

状态、工具与重置语义。 对于每个保留环境，我们联合构建三类资产：(1) 描述实体、关系和初始记录的数据库或结构化 fixture，用于实例化 $\mathcal{S}$ 与 ρ_0 ；(2) 与 MCP schema 对齐的工具实现，其中一次工具调用执行读取或写入并实现转移 T ；(3) 面向模型的工具描述，用于定义 $\mathcal{A}_{\text{tool}}$ 与返回观察 $\mathcal{O}$ 。每个 episode 从规范 fixture 创建独立的内存实例，写操作只改变该实例而不回写规范数据，因此不同 rollout 相互隔离且可由相同种子重置。相反，同一 episode 内所有 Worker 连接同一个工具执行器并共享可变速态；这使前序写入、并发竞争和后续核验成为真实的环境交互，而不是在提示中模拟的依赖关系。

可执行性检查。 环境进入 World Bank 前依次通过 schema 解析、实现加载、工具调用和重置一致性检查。对于读工具，我们检查其返回值能够由初始 fixture 支持；对于写工具，我们检查目标记录确实发生预期变化，并在重置后恢复；异常、超时和非法参数必须产生结构化且可复现的错误观察。该过程同时剔除接口-实现不一致以及依赖不可控外部状态的环境，为后续程序化验证提供稳定执行语义。

4.2 原子任务与局部验证器

在环境 E 中，一个原子任务是能够由单个 Worker 独立完成、且具有明确成功条件的最小任务单元。我们将其记录为

$$u_j = (q_j, \mathcal{A}_j, y_j^*, V_j^{\text{atom}}, \Gamma_j), \quad \Gamma_j = (\text{Read}_j, \text{Write}_j, \text{Pre}_j, \text{Post}_j), \quad (5)$$

其中 q_j 是自包含的自然语言指令， $\mathcal{A}_j \subseteq \mathcal{A}_{\text{tool}}$ 是完成该任务所需的工具子集， y_j^* 是结构化标准答案， V_j^{atom} 是确定性代码验证器。 Γ_j 描述任务对状态对象的读集合、写集合、执行前必须成立的条件以及成功后的状态事实；它只保存在生成侧元数据中。原子任务既包含从环

境中检索、筛选和聚合信息的只读目标，也包含创建、更新或删除对象的写入目标，从而使组合任务能够同时覆盖数据流依赖与共享状态约束。

标准答案由规范初始状态上的可执行程序生成，而不是直接依赖语言模型判断。验证器首先检查最终 JSON 的字段、类型和值；对于产生状态变化的任务，还检查目标对象及相关约束在最终状态中是否成立。当 u_j 被组合进复杂任务 x 时，我们由同一成功条件实例化训练期局部验证器 $V_{x,j}^{\text{proc}}$ ，但其输出只用于第 5 节的过程奖励，不返回给 Orchestrator。每个任务还通过正例回放与反事实测试： y_j^* 必须通过验证，而删除必需字段、替换关键值或撤销目标状态变化后必须失败，以过滤恒真或只检查答案格式的验证器。

4.3 依赖与冲突建模

为了从原子任务恢复可组合结构，我们对工具实现、输入输出 schema 和状态表进行静态分析，得到 Γ_j 的候选标注，再通过受控执行比较调用前后的状态与返回值进行确认。由此区分两类关系。若 u_j 消费 u_i 的输出，或 u_i 的后置条件是 u_j 的必要前置条件，则加入有向依赖边

$$(u_i, u_j) \in \mathcal{D}_x \iff y_i^* \rightsquigarrow q_j \text{ 或 } \text{Post}_i \models \text{Pre}_j, \quad (6)$$

其中 $\rightsquigarrow$ 表示前一节点的结构化结果被绑定为后一节点的参数。对于不存在已知先后关系的节点，我们计算共享状态重叠

$$\chi_{ij} = \mathbb{I}[\text{Write}_i \cap (\text{Read}_j \cup \text{Write}_j) \neq \emptyset \vee \text{Write}_j \cap (\text{Read}_i \cup \text{Write}_i) \neq \emptyset]. \quad (7)$$

当 $\chi_{ij} = 1$ 且并发执行不能保证与某一合法串行顺序等价时，将无序边 $\{u_i, u_j\}$ 加入 $\mathcal{C}_x$ 。反之，读写集合不相交且没有数据依赖的节点可作为并行候选。静态分析只用于提出候选关系；最终标注还需由两种执行顺序及并发执行的状态差异确认，以避免仅凭工具名称推断出伪依赖。

4.4 复杂编排任务合成

组合器仅使用串行组合 Seq 与并行组合 Par 两个结构算子递归构造复杂任务。串行组合将上游输出或状态事实绑定到下游参数；并行组合合并不存在数据依赖的任务分支。通过两个算子的多层嵌套，组合器可生成长依赖链、宽并行结构、多轮 fork-join、非对称分支及汇合节点。基于生成的结构。

对于合成任务 x ，局部成功条件记为 $\phi_j(s, y)$ ，跨节点的一致性约束记为 $\phi_{\text{cross}}(s, y)$ 。我们独立编译严格终局验证器

$$V_x^{\text{final}}(s, y) = \mathbb{I}\left[\bigwedge_{j=1}^m \phi_j(s, y) \wedge \phi_{\text{cross}}(s, y)\right], \quad (8)$$

使“每个分支分别给出合理答案、但最终遗漏字段或破坏共享状态”的轨迹仍被判为失败。生成器同时保存 $G_x = (\mathcal{U}_x, \mathcal{D}_x, \mathcal{C}_x)$ 和一个满足依赖、冲突及资源约束的 Oracle 调度；运行时仅向策略提供 $(q_x, \mathcal{A}_{\text{tool}})$ 以及实际执行反馈。

4.5 难度控制与质量过滤

除了节点数 m ，任务难度还由依赖深度、最大宽度、fork-join 次数、分支长度方差、冲突密度和恢复事件数共同控制。为刻画一个任务在并发预算 K 下的可并行程度，令 $W^*(x)$ 为 Oracle 完成全部节点所需的总工具工作量， $C_K^*(x)$ 为满足 $\mathcal{D}_x$ 、 $\mathcal{C}_x$ 和并发上限 K 时的最短关键路径，定义

$$P_K(x) = 1 - \frac{C_K^*(x)}{W^*(x)}. \quad (9)$$

$P_K(x)$ 接近 0 表示额外 Worker 几乎不能缩短关键路径，数值越大则表示可安全重叠的工作越多。我们联合以上结构统计分层采样，避免训练集被纯并行或纯串行任务主导；数据划分按环境隔离，并另外保留未见工具组合、未见图形和更长依赖链作为结构泛化集合。

最后，所有样本经过四级过滤：(1) 环境级检查工具可执行、状态可重置；(2) 节点级检查原子 Oracle 与局部验证器的一致性；(3) 图级检查 DAG 无环、依赖可满足、并行节点无未标注冲突，且交换冲突节点时能够观察到预期差异；(4) 任务级运行 Oracle 轨迹并要求

$V_x^{\text{final}} = 1$ ，同时用遗漏节点、违反依赖和冲突写入的反例确认严格验证器能够拒绝投机解。真实图、Oracle 与局部验证结果在发布给策略前全部移除，只保留训练奖励服务所需的隐藏索引。现有运行时已经支持 episode 级内存重置、Worker 间共享状态与确定性终局判分；仅带工具路径聚合统计、但缺少 Γ_j 与完整 G_x 的记录不被视为通过上述图级过滤的组合任务。

5 两阶段编排学习

上一节给出了带隐藏执行图和确定性验证器的组合任务。本节说明如何在不向策略泄露真实依赖图 G_x 的前提下，先获得可靠的工具执行器，再学习根据任务与运行时反馈动态构造执行 DAG H_τ 的 Orchestrator。两阶段使用彼此分离的数据和优化目标：第一阶段只学习环境工具动作，第二阶段冻结 Worker，仅更新编排策略。

5.1 阶段一：学习可靠的 WORKER

我们首先在基础任务集合 $\mathcal{D}_w$ 上训练 Worker π_w 。对于可执行任务单元 u ，Worker 接收目标 q 与工具 schema，从可重置状态 s_0 出发，生成交替包含工具动作和环境观察的线性轨迹

$$\tau_w = (q, a_1, o_1, \dots, a_T, o_T, y_T). \quad (10)$$

其奖励由任务单元的确定性验证器给出：只有最终答案 y_T 与终止状态 s_T 同时满足验证约束时才记为成功。该阶段不提供编排动作，也不奖励拆分或并发，因此优化对象仅是 schema 遵循、多轮工具调用、状态读取与局部错误修正等 Executor 能力。

训练完成后，我们依据验证集的严格成功率选择 Worker checkpoint，并在后续 Orchestrator 训练和所有 Multi-Agent 对照中固定该 checkpoint。不同方法由此共享相同的底层执行能力；性能差异主要来自任务拆分、委派和并发结构，而不是 Worker 在第二阶段继续更新。

5.2 阶段二：学习 ORCHESTRATOR

第二阶段使用组合任务集合 $\mathcal{D}_o$ 。每条训练记录包含任务 x 、可重置环境、终局验证器 V_x^{final} ，以及仅供任务生成和奖励计算使用的 G_x 与局部验证器 $\{V_{x,j}^{\text{proc}}\}_{j=1}^m$ 。Orchestrator 的实际输入只包含任务文本、工具 schema、历史观察和剩余预算 b_t ； G_x 、读写集合、冲突边和局部验证结果均不写入 prompt 或 observation。

在时刻 t ， π_o 从式 4 的动作空间采样动作。act 让 Orchestrator 直接调用任务工具；create subagent 注册 Worker 的角色、局部指令与可见工具子集；assign task 将一项工作交给指定 Worker，并将其返回连接到后续决策；assign tasks 同时派发一组任务，在并发上限 K 内形成 fork-join；finish 提交最终答案并终止 episode。依赖后的补充执行或失败分支重做仍通过新的 assign task/assign tasks 实现，而不额外引入动作类型。

因此，一次 rollout 并非单一模型序列。Orchestrator 的决策节点、各 Worker 的工具调用、共享状态转移和 assign tasks 的汇合点共同形成动态执行 DAG H_τ 。同一 episode 中的 Worker 共享环境状态，但各自保留独立上下文；批量委派的结果全部返回后，Orchestrator 再决定聚合、继续委派或结束。Worker 生成、工具观察与委派结果作为环境反馈进入 Orchestrator 历史，但其 token 的 loss mask 为零；只有 Orchestrator 生成的推理 token 与编排动作参与第二阶段的策略更新。

5.3 编排训练数据与课程

$\mathcal{D}_w$ 由基础可执行任务构成， $\mathcal{D}_o$ 则由同一任务库按第 4 节的依赖与冲突规则组合而成。为防止模型把工具名或固定 workflow 当作执行图，我们在划分数据时同时隔离环境、工具组合与 DAG 形状，并对自然语言表述和同构图进行扰动。对于同一输入采样的一组强化学习轨迹，每条轨迹均从独立重置的 s_0 开始，避免并行 rollout 之间共享状态。

训练课程先使用结构差异明确的短任务：Serial 样本提供错误并发的负例，Parallel 样本暴露不必要等待的代价；随后加入 Mixed、Contention 与 Adaptive Recovery，并逐步增加依赖深度、分支长度方差和冲突密度。每个 batch 按并行潜力 $P_K(x)$ 与任务类型分层采样，同时随机采样并发预算 K 、关键步数预算 B 和总工作量预算 W_{max} 。由此得到的 budget-conditioned policy 可在同一组参数下学习不同资源约束，而不是将某个固定并行度写入系统提示。

5.4 执行步骤感知的可验证奖励

严格任务奖励。 令 s_T 和 y_T 分别为执行结束时的共享环境状态与提交答案。终局奖励定义为

$$R_{\text{task}}(\tau; x) = \mathbb{I}[a_T = \text{finish} \wedge V_x^{\text{final}}(s_T, y_T) = 1]. \quad (11)$$

该奖励要求所有子目标、答案字段和跨分支状态约束同时成立；局部完成率不能替代严格成功。

步骤级过程奖励。 稀疏的终局奖励难以区分“分解合理但某一支失败”和“从未形成有效执行计划”的轨迹。为此，我们只在训练期利用局部验证器构造执行步骤感知的 shaping 信号。记 $y_{j,t}$ 为截至事件 t 针对子任务 u_j 产生的候选结果，并定义

$$z_{j,t} = \mathbb{I}[\exists t' \leq t : V_{x,j}^{\text{proc}}(s_{t'}, y_{j,t'}) = 1], \quad c_t = \frac{1}{m} \sum_{j=1}^m z_{j,t}. \quad (12)$$

$z_{j,t}$ 采用首次通过后的单调记录，因此同一子任务重复完成不会重复得分。进一步令 d_t 为截至 t 对已验证子任务产生的累计重复工作量， e_t 为累计无效编排事件数，包括非法动作、违反前置条件的派发和触发确定性冲突的并发。过程奖励为

$$R_{\text{proc}}(\tau; x) = \sum_{t=1}^T \gamma_p^{t-1} [\alpha(c_t - c_{t-1}) - \beta(d_t - d_{t-1}) - \xi(e_t - e_{t-1})], \quad (13)$$

其中 $c_0 = d_0 = e_0 = 0$ ， $\gamma_p \in (0, 1]$ 是过程折扣， $\alpha, \beta, \xi \geq 0$ 分别控制有效进展、重复工作与无效编排事件的权重。局部验证器及上述统计均位于 reward side，绝不作为 Orchestrator 的 observation；推理时也不需要它们。 R_{proc} 只改善信用分配，不改变式 11 对成功的定义。

5.5 关键步数与总工作量预算

仅最小化总调用数会错误惩罚有效并行，而仅最大化并发又可能用大量重复工作换取较短等待时间。我们因此同时约束逻辑延迟与总工作量。根据 H_τ 中的 fork-join 边，将执行划分为 L 个同步阶段；令 $\mathcal{I}_\ell$ 为阶段 ℓ 的 Worker 分支集合， $S_o^{(\ell)}$ 为该阶段的 Orchestrator 编排动作与直接工具动作数， $S_{w,i}^{(\ell)}$ 为分支 i 的工具动作数。参照并行 Agent 系统中的逻辑关键步数定义 (Kimi Team, 2026)，Critical Tool Steps 写为

$$\text{CTS}(\tau) = \sum_{\ell=1}^L \left(S_o^{(\ell)} + \max_{i \in \mathcal{I}_\ell} S_{w,i}^{(\ell)} \right), \quad (14)$$

其中无 Worker 的阶段令最大项为零。式 14 在每个并行阶段内取最长分支，再跨阶段求和，因而是同步式实现中执行 DAG 最长加权路径的等价形式，而非整个 episode 只取一次最长 Worker。

相应地，总工作量累加所有实际执行分支：

$$W(\tau) = \sum_{\ell=1}^L \left(S_o^{(\ell)} + \sum_{i \in \mathcal{I}_\ell} S_{w,i}^{(\ell)} \right). \quad (15)$$

CTS 使用 max 衡量逻辑等待路径， W 使用求和衡量系统付出的总动作成本；两者共同排除“复制多个 Worker 执行相同任务”的伪并行。训练回报写为

$$R(\tau; x, b) = R_{\text{task}}(\tau; x) + \eta R_{\text{proc}}(\tau; x) - \lambda \left[\frac{\text{CTS}(\tau)}{B} - 1 \right]_+ - \mu \left[\frac{W(\tau)}{W_{\text{max}}} - 1 \right]_+, \quad (16)$$

其中 $[z]_+ = \max(z, 0)$ ， $b = (B, K, W_{\text{max}})$ ， $\eta, \lambda, \mu \geq 0$ 分别控制过程塑形、逻辑延迟和总工作量。环境同时执行硬预算：峰值并发不得超过 K ，CTS 或总工作量越界时立即终止，且该轨迹的严格任务奖励为零。预算越界的最后一个事件仍写入日志，以使式 16 的惩罚项提供方向性信号。

5.6 组相对策略优化

我们采用 Group Relative Policy Optimization (GRPO) (Shao et al., 2024) 优化两个阶段的可验证回报。第二阶段对每个 (x, b) 从旧策略采样 G 条独立轨迹, 令 R_i 为式 16 的回报, 并计算组内标准化优势

$$\hat{A}_i = \frac{R_i - \text{mean}_j(R_j)}{\text{std}_j(R_j) + \epsilon}. \quad (17)$$

记 $m_{i,t} \in \{0, 1\}$ 为 token loss mask, $\rho_{i,t}(\theta) = \pi_\theta(a_{i,t} | h_{i,t}) / \pi_{\theta_{\text{old}}}(a_{i,t} | h_{i,t})$, 则最大化

$$J_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{\sum_t m_{i,t}} \sum_t m_{i,t} \min \left(\rho_{i,t}(\theta) \hat{A}_i, \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon_l, 1 + \epsilon_h) \hat{A}_i \right) \right]. \quad (18)$$

其中 ϵ_l 与 ϵ_h 是重要性比率的上下裁剪范围。对 Orchestrator 轨迹, 只有其自身生成 token 满足 $m_{i,t} = 1$; 冻结 Worker 的生成、环境工具返回和协调器 observation 均取零。第一阶段使用同一优化形式, 但将回报替换为基础任务验证奖励, 并仅训练 Worker 的模型 token。

5.7 训练与推理流程

每次训练迭代首先按结构分桶采样任务和预算, 重置环境并生成一组动态执行 DAG; 随后由终局验证器、训练期局部验证器及事件日志计算式 16, 最后仅对相应阶段的可训练 token 执行式 18 的更新。推理时移除所有局部验证器与过程奖励接口, Orchestrator 只能依据任务、工具 schema、Worker 返回、共享状态观察和剩余预算在线构造 H_τ , 最终通过 finish 提交答案。

实现边界。 当前代码资产已经支持基础任务上的 Worker GRPO、冻结 Worker 的独立服务、仅更新 Orchestrator 的 token mask、动态创建角色、串行与并发委派、episode 内共享状态以及确定性终局判分; Multi-Agent 路径已完成 CPU 端到端 dry run。当前原型的 Orchestrator 仅暴露 create subagent/assign task/assign tasks/finish, 且默认要求至少委派一次。论文方法中的 act、无强制委派、组合任务的完整隐藏标注、步骤级过程奖励、事件 DAG 上的 CTS/W 统计和预算条件化训练仍属于主实验实现目标; 在这些组件及完整 Multi-Agent GPU 训练完成前, 本文不据此主张训练效果。

6 实验

本节围绕 Introduction 中的三项主张设计实验: 合成环境能否训练出可靠的工具型 Worker, 学习型 Orchestrator 能否在未知任务结构上同时提高正确性与执行效率, 以及这种编排能力能否迁移到外部长程任务和不同模型规模。所有正式结论均基于固定数据版本、统一资源预算和多随机种子评测; 当前尚未完成的结果以占位符明确标出。

6.1 评测任务

我们使用三组互补任务, 分别隔离基础执行能力、现实长程能力和可控的编排结构。

基础工具调用。 BFCL (Patil et al., 2025) 覆盖单轮、多轮、并行函数调用及幻觉检测, 用于评估工具选择、参数生成和格式遵循; τ^2 -Bench (Barres et al., 2025) 在有状态的用户-Agent-工具交互中评估多轮任务完成能力。二者主要回答 Worker 训练是否改善通用工具执行, 以及冻结 Worker 后的编排训练是否造成基础能力退化; 它们不单独支撑任务编排结论。

长程任务编排。 我们采用 DeepPlanning、Wide Search 与 Toolathlon (Li et al., 2025)。DeepPlanning 强调长依赖链、跨步骤约束和状态维护, Wide Search 强调多分支检索、证据聚合和可并行覆盖, Toolathlon 则包含跨工具、跨应用且执行路径较长的现实任务。对于每个套件, 我们使用其官方任务与成功判定, 并通过统一适配层记录 Orchestrator/Worker 事件、工具调用、token 和真实时间。若原任务不存在可靠的中间结构标注, 则只使用官方终局指标, 不从模型自述反推依赖图。

Table 1: 评测套件及其在实验中的作用。外部 benchmark 使用官方主指标；只有具备完整事件日志的任务才报告 CTS 与 Total Work。

套件	主要检验能力	主报告指标
BFCL	函数选择、参数与格式	官方 accuracy
τ^2 -Bench	有状态多轮工具交互	官方 task success
DeepPlanning	长依赖链与状态维护	success、steps、time
Wide Search	分支覆盖与证据聚合	success、CTS、time
Toolathlon	跨工具长程执行	官方 score、time
MAW held-out	隐藏依赖、冲突与恢复	strict success、CTS、W

MAW 受控测试集。 第 4 节生成的 held-out 任务提供 Serial、Parallel、Mixed、Contention 和 Adaptive Recovery 五类结构，以及仅供评测侧使用的 G_x 、局部验证器和 Oracle 调度。该集合使我们能够在相同语义难度下控制依赖深度、最大宽度、冲突密度与并行潜力 $P_K(x)$ ，并直接诊断错误并行、无效等待和恢复策略。训练、验证与测试按环境 ID 隔离，另设未见工具组合、未见 DAG 同构类和超训练长度三个 OOD 切片。

[待补: 评测版本: 在正式实验前固定 BFCL、 τ^2 -Bench、DeepPlanning、Wide Search 和 Toolathlon 的 commit、任务子集、官方 metric 与许可证；对不支持并发的原生环境明确只评估 Single-Agent/Worker 能力。]

6.2 模型、训练设置与比较方法

模型设置。 主实验以 Qwen3-8B 为基础模型。阶段一在原子任务上训练 Worker，并依据 held-out 基础任务的严格成功率选择唯一 checkpoint；阶段二复制该 checkpoint 初始化 Orchestrator，冻结 Worker，仅更新 Orchestrator token。所有 Multi-Agent 方法共享相同 Worker、工具 schema、上下文上限与解码配置。模型规模实验进一步交叉替换 Orchestrator 与 Worker，以区分规划能力和执行能力的贡献。

执行策略。 我们比较以下系统：(1) **Single-Agent ReAct**，禁止委派，由单个模型完成全部工具调用；(2) **Serial Multi-Agent**，允许拆分但同一时刻只运行一个 Worker；(3) **Eager Fully-Parallel**，把所有当前判断为 ready 的任务尽快派满至 K ；(4) **Prompt-only Adaptive**，使用与本文相同的动作空间和预算输入，但不进行 Orchestrator RL；(5) **Learned Adaptive Orchestrator**，使用第 5 节的完整训练目标；(6) **Oracle-ASAP**，只在 MAW 受控任务上读取真实 G_x ，作为不可部署的关键路径参照。

为隔离编排贡献，Serial、Eager、Prompt-only 与本文方法使用同一个 Worker checkpoint、任务分解提示模板和结果聚合格式；除 Oracle-ASAP 外，任何方法都不能观察隐藏图、读写集合、局部验证器输出或 Oracle span。Single-Agent ReAct 与 Multi-Agent 方法具有相同的总工作量、token 和超时上限，不能通过额外预算获得不公平优势。

6.3 指标、预算与统计协议

严格成功率与逻辑延迟。 外部套件同时报告官方指标；在可执行组合任务上，以式 11 的二值终局判定作为主准确率。给定预算 $b = (B, K, W_{\max})$ ，定义预算内成功率

$$A_{\pi}(B, K, W_{\max}) = \frac{1}{N} \sum_{n=1}^N \mathbb{I}[R_{\text{task}}(\tau_n; x_n) = 1, \text{CTS}(\tau_n) \leq B, W(\tau_n) \leq W_{\max}]. \quad (19)$$

超过任一硬预算的轨迹立即终止并记为失败，而非从统计中剔除。对于具有隐藏图与 Oracle span 的 MAW 受控任务，我们扫描

$$K \in \{1, 2, 4, 8\}, \quad B \in \{1.0, 1.5, 2.0, 3.0\} \times C_K^*(x), \quad (20)$$

并在每个 (K, B) 下统一 $W_{\max}$ ，绘制 Strict Success-CTS Pareto 前沿。外部 benchmark 不使用测试任务的 Oracle 信息，而是在验证集上预先冻结一组绝对 B 与 $W_{\max}$ ，再原样应用于测试集。

Table 2: 基础工具调用与长程任务主结果。数值均使用各套件官方指标；“-”表示正式复跑尚未完成。

方法	BFCL	τ^2	DeepPlan.	Wide Search	Toolathlon
Qwen3-8B Base	-	-	-	-	-
+ Worker RL	-	-	-	-	-
+ MAW Orchestrator	-	-	-	-	-

Table 3: MAW held-out 的编排主结果。最终表将在预先指定的公共预算点报告；“-”表示待正式实验。

方法	Strict $\uparrow$	CTS $\downarrow$	Time $\downarrow$	Work $\downarrow$
Single-Agent ReAct	-	-	-	-
Serial Multi-Agent	-	-	-	-
Eager Fully-Parallel	-	-	-	-
Prompt-only Adaptive	-	-	-	-
MAW Orchestrator	-	-	-	-
Oracle-ASAP	-	-	-	-

成本与诊断指标。 主延迟指标为式 14 的 CTS，并同时报告 wall-clock p50/p90。成本指标包括式 15 的 Total Work、主/副 Agent token、模型调用数、峰值并发、handoff 数与失败后的重派次数。局部完成率、字段级准确率、依赖违反率、冲突重叠率和重复工作率仅用于错误解释，不能替代严格终局成功率。

公平性与不确定性。 所有方法复用相同任务、初始状态、工具后端、模型服务、温度、上下文、硬件与并发槽位。每个任务使用配对随机种子；成功率报告 bootstrap 95% 置信区间和配对二元检验，延迟及成本报告中位数、p90 与配对置信区间。Pareto 比较除曲线外，还报告 Accuracy-Budget AUC、达到固定成功率所需的最小 CTS，以及固定 CTS 下的最高成功率。wall-clock 计时排除模型/环境冷启动，并单独披露模型服务与工具服务配置。

6.4 基础工具能力与长程任务结果

表 2 对应 Introduction 中的通用 benchmark 主张。Base 与 Worker 的差异衡量阶段一训练；Worker 与 MAW 的差异检查第二阶段是否保留基础执行能力。长程套件同时比较 Single-Agent 和完整编排系统。

[待补: 主结果: 填入统一版本下的 BFCL、 τ^2 -Bench、DeepPlanning、Wide Search 与 Toolathlon 分数及 95% CI; 同步报告对应 wall-clock。只有当所有预注册比较均支持时，才能将 Introduction 中的“SOTA”与“大幅领先”替换为具体数值。]

6.5 编排策略的准确率-延迟比较

MAW held-out 是隔离任务编排收益的核心实验。表 3 在同一预算点比较各策略，图 3 则扫描 B 与 K 展示完整前沿。总体结果之外，我们分别报告五类任务结构；预期的合理行为不是在所有任务上最大化并发，而是在 Serial 上接近单线程执行、在 Parallel 上接近 Eager、并在 Mixed/Contention/Recovery 上形成更好的折衷。

[待补: Pareto 结论: 补入 Accuracy-Budget AUC、固定成功率下的最小 CTS、固定 CTS 下的最高成功率，以及 Total Work 与 token 的配套变化；不得用图 1 中的概念点位作为实验结果。]

6.6 模型规模、结构泛化与预备证据

Orchestrator/Worker 规模矩阵。 我们在相同工具、任务和预算下交叉组合不同规模的主 Agent 与冻结 Worker。该实验回答提升来自更强的全局规划还是更强的局部执行，并检验小 Orchestrator 调度大 Worker、以及大 Orchestrator 调度小 Worker 的性价比。闭源模型只作为预算可对齐时的参考系统，并同时报告其 token、模型调用和真实时间。

[待补：主结果图：在多个 B 与 K 下绘制各方法的 Strict Success-CTS Pareto frontier 与 95% CI；分面展示总体、Serial/Parallel、Mixed/Contention/Recovery 和 OOD。]

Figure 3: 学习型编排与固定执行拓扑的准确率-逻辑延迟比较。Oracle-ASAP 仅提供环境关键路径参照。

Table 4: Orchestrator/Worker 模型规模分析占位。模型集合与闭源版本将在实验冻结时确定。

Orchestrator	Worker	Strict	CTS	Time	Work
Qwen3-8B	Qwen3-8B	—	—	—	—
small	Qwen3-8B	—	—	—	—
Qwen3-8B	small	—	—	—	—
large	Qwen3-8B	—	—	—	—
Qwen3-8B	large	—	—	—	—
Closed-source ref.	matched	—	—	—	—

[待补：规模设置：冻结 small/large 与闭源参考模型的具体名称、版本、API 日期、解码配置及等预算规则；补入各组合的多种子结果与置信区间。]

结构与长度泛化。 除环境内随机划分外，我们在未见环境、工具组合、DAG 同构类和更长任务上分别报告结果，并按 $P_K(x)$ 、深度、宽度和冲突密度分桶。该设计用于判断模型是否学习了可迁移的拓扑决策，而非记住工具名或固定 workflow。

[待补：泛化结果：补入 IID 与四类 OOD 切片的 Strict Success、CTS、Total Work 和错误类型；同时给出随节点数、依赖深度和宽度增长的 scaling 曲线。]

已有 Worker 预备结果。 表 5 汇总 0716 记录中可复核的 Qwen3-8B 单 Agent 数值。Epoch 1 在 DeepPlanning 与 Shopping 上较 Base 提升，BFCL 基本持平；Epoch 2 的 BFCL 下降，说明简单延长训练并不稳定。这些结果只用于选择和复核 Worker checkpoint，不能支撑 Multi-Agent 或 Pareto 结论。

当前 Multi-Agent 代码只完成 CPU 端到端 dry-run，验证了冻结 Worker、并发委派、共享 episode 状态、actor-only loss mask 与确定性终局判分链路；尚无可复核的 Multi-Agent GPU optimizer step、CTS、wall-clock 或 Pareto 数值。因而表 2-4 与图 3 的空缺必须由正式多种子实验补齐，不能由上述 Worker 预备结果外推。

7 分析、消融与局限

主实验回答学习型 Orchestrator 能否改善严格成功率-CTS 的 Pareto 前沿；本节进一步分析这种改善是否来自对任务结构的有效恢复，而非更多 Worker、更多工具调用或更强的底层执行器。由于真实任务图 G_x 只保存在生成与评测侧，以下结构分析均为离线诊断，不向策略提供额外信息。

7.1 编排行为分析

我们将 rollout 形成的执行 DAG H_τ 与隐藏结构 $G_x = (\mathcal{U}_x, \mathcal{D}_x, \mathcal{C}_x)$ 对齐，但不要求两张图逐边相同：同一任务可能存在多个等价的合法调度。具体报告四组性质级指标：（1）依赖满足，即下游分支是否在其前置结果或状态事实就绪后才开始；（2）冲突规避，即冲突边两端是否发生不安全重叠；（3）并行利用，即无依赖、无冲突的 ready 节点在被派发前等待了多久；（4）协调开销，包括子任务粒度、Worker 利用率、同步次数、交接次数与重复工作率。它们分别区分“并行过早”“并行不足”和“拆分过细”三类行为。

Table 5: 已有 Qwen3-8B Worker 预备结果；投稿前需在冻结版本上统一复跑。

模型	BFCL	DeepPlanning	Shopping	Travel
Qwen3-8B Base	70.54	2.50	5.00	0.00
Worker Epoch 1	70.36	4.58	9.17	0.00
Worker Epoch 2	65.92	4.58	9.17	0.00

进一步地，以第 4 节的 Oracle 关键路径 $C_K^*(x)$ 为参照，定义成功轨迹的归一化关键路径遗憾

$$\text{SpanRegret}_K(x, \tau) = \frac{\text{CTS}(\tau) - C_K^*(x)}{C_K^*(x)}. \quad (21)$$

严格成功率衡量最终任务是否完成，而 SpanRegret_K 衡量完成任务时的调度质量；二者必须联合解读，避免把快速失败误判为高效。我们还按同步阶段可视化典型 H_τ ，检查模型是否形成“安全分叉-依赖等待-结果汇合-局部恢复”的动态拓扑。

[待补：结果：补入 Learned Adaptive、Serial Multi-Agent 与 Eager Fully-Parallel 的依赖违反率、冲突重叠率、ready 节点等待、重复工作率及 SpanRegret ；给出至少一个成功案例和一个失败案例的执行 DAG。只有这些指标与终局收益一致时，才能声称模型学到了任务结构相关的编排行为。]

7.2 关键消融

我们围绕第 5 节的信用分配、成本约束与训练分布进行以下消融，并保持 Worker checkpoint、训练任务数与推理预算不变。

- **无步骤级过程奖励**：令 $\eta = 0$ ，仅保留严格终局奖励与预算惩罚，用于检验局部验证器提供的信用分配是否改善稀疏奖励下的学习，而非改变成功定义。
- **单一成本约束**：分别移除 CTS 惩罚和 Total Work 惩罚。前者检验策略是否转向更长的保守轨迹，后者检验低逻辑延迟是否由冗余派发和重复查询换取。
- **无预算条件化**：从策略输入中移除 $b = (B, K, W_{\max})$ ，在固定预算上训练和测试，以判断同一策略能否随资源约束调整委派粒度和并行度。
- **简化任务结构**：仅使用 Serial/Parallel 组合训练，移除 Mixed、Contention 与 Adaptive Recovery；该消融检验复杂组合及课程学习是否是获得同步、冲突规避和局部重派能力的必要条件。
- **不冻结 Worker**：在第二阶段同时更新 Worker 与 Orchestrator，作为归因诊断。若该设置提升最终性能，却同时改变基础工具任务成功率，则不能把收益单独归因于编排策略。

除严格成功率与 CTS 外，每项消融同时报告 Total Work、过程完成曲线和对应的行为指标；因此，即使两个模型具有相同的最终成功率，也能区分其信用分配速度与资源使用方式。

[待补：结果：补入完整模型与上述消融在总体及 Mixed/Contention/Recovery 子集上的结果，报告均值、95% 置信区间和相对变化；重点说明过程奖励、CTS 与 Total Work 是否分别产生互补作用。]

7.3 按任务结构与错误类型分析

为避免总体平均掩盖固定拓扑的适用范围，我们按 Serial、Parallel、Mixed、Contention 和 Adaptive Recovery 分组，并进一步按并行潜力 $P_K(x)$ 、依赖深度、最大宽度、冲突密度与分支长度方差分桶。结构分析关注学习型策略是否在低 P_K 任务中避免无效委派，在高 P_K 任务中减少 ready 节点等待，并在 Mixed/Contention 任务中同时控制依赖错误和关键路径。这里比较的是随任务结构变化的趋势，而不是用纯并行子集上的速度优势代表整体编排能力。

失败轨迹依据验证器与事件日志划分为八类：**decomposition**（遗漏子目标或边界错误）、**dependency**（前置条件未满足）、**contention**（共享状态冲突）、**handoff**（必要参数或证据未传递）、**aggregation**（局部结果正确但最终合并错误）、**recovery**（可恢复失败后未重派，或不必要地全局重做）、**budget**（CTS 或总工作量耗尽）和 **executor**（Worker 工具调用或局部

推理失败)。能够由隐藏图、代码验证器和事件时间戳确定的类别采用程序化归因，其余案例再行人工复核；模型自身的解释不作为错误标签。

[待补：结果：报告各结构桶的 Strict Success-CTS 曲线及八类错误占比；说明随着任务深度、宽度和冲突密度增加，主要瓶颈如何从 Executor、分解或交接转移到依赖、冲突、聚合或预算。]

7.4 效率、模型规模与扩展性

我们从任务规模和系统规模两条轴考察扩展性。任务侧逐步增加原子节点数、依赖深度和最大宽度，系统侧扫描并发上限 K ；每个点联合报告严格成功率、CTS、Total Work、token 用量、峰值并发及 wall-clock p50/p90。为判断新增 Worker 是否产生有效加速，我们同时报告相对 Serial Multi-Agent 的逻辑加速比与并行效率，并检查当 K 超过任务宽度后，收益是否被派发、同步和状态竞争开销抵消。

由于 CTS 只计逻辑关键步而不建模不同工具的真实耗时，我们计算其与 wall-clock 延迟的秩相关性，并按工具类型、任务规模和 K 分层。若相关性在某些环境或高并发区间显著下降，则对应结论应以真实延迟为准，而不能由 CTS 外推。

最后，我们交叉替换不同规模的 Orchestrator 与冻结 Worker，在相同工具接口、上下文、并发和总工作量预算下区分“更强的规划模型”与“更强的执行模型”分别带来的收益。与闭源系统的比较仅在任务输入、工具可见性与预算能够对齐时报告，并同时披露模型与系统成本。

[待补：结果：补入随节点数、深度、宽度和 K 变化的扩展曲线、CTS-wall-clock 相关系数，以及 Orchestrator/Worker 模型规模矩阵。若统计结果支持，再给出相对闭源模型的准确率-延迟结论；否则删除 Introduction 中“远超闭源模型”等表述。]

7.5 局限

首先，当前直接证据仍限于 Worker 的预备评测与 Multi-Agent CPU dry-run；它们验证了基础 Executor 训练、冻结 Worker、并发委派、loss mask 和确定性判分链路，但不能证明 Orchestrator 训练有效或 Pareto 前沿得到改善。完整的 Multi-Agent GPU 训练、多随机种子评测和上述消融完成前，本文不据此主张 SOTA。

其次，真实结构 G_x 是生成器给出的一个可执行解释，而非任务唯一的语义分解；现实请求还可能含有无法由读写集合穷举的隐式依赖。因此我们使用性质级行为指标而非图的精确匹配，但由合成环境得到的结构泛化仍需在真实工具任务上验证。训练期局部验证器也依赖可程序化的中间条件，这限制了方法对开放式目标的直接适用性。

第三，CTS 是可复现的逻辑延迟代理，不等于用户感知时间。异质工具耗时、服务排队、网络抖动和硬件并行效率都会改变 wall-clock 结果；当前同步式 fork-join 实现也尚未覆盖完全异步或递归嵌套的执行图。第四，World Bank 主要由可重置的数据库/API 类 fixture 构成，与网页、终端、人机协作及具有不可逆外部副作用的真实 MCP 服务仍有差距；真实部署还需要权限隔离、事务控制和安全审计。

最后，冻结同质 Worker 虽有利于隔离编排贡献，却没有研究异质模型选择、技能路由、Worker 间直接通信与多层 Orchestrator。更大并发度也会增加 token、算力和服务成本，即使它降低了关键路径；因而本文的准确率-延迟 Pareto 结论不能替代面向具体部署约束的总体成本分析。

8 结论

本文关注长程工具任务中区别于基础工具调用的任务编排能力，并提出三阶段训练框架 MULTI-AGENT WORLD。我们首先从有状态 MCP 描述构建可执行、可重置且带代码验证器的 World Bank，再依据原子任务的读写集合、前后置条件与共享状态冲突合成 Serial、Parallel、Mixed、Contention 和 Adaptive Recovery 等复杂编排任务，最后采用两阶段训练：先学习可靠的工具型 Worker，再冻结 Worker 并训练 Orchestrator 在线构造串并行混合的执行 DAG。除严格终局奖励外，训练还利用局部验证器提供执行步骤感知的过程奖励，并以 CTS 和 Total Work 分别约束逻辑关键路径与系统总工作量。

评测协议在统一模型、Worker、工具和资源预算下联合报告严格任务成功率、CTS、真实完成时间与总工作量，从而将“完成得是否正确”和“需要等待与付出多少”置于同一准确率-延迟 Pareto 框架中。现有证据只确认了 Worker 训练及 Multi-Agent CPU dry-run 链路，尚不足以支持性能或 SOTA 结论。**[待补：结果：正式实验完成后，在此填入 MAW 相对 Single-Agent ReAct、Serial Multi-Agent、Eager Fully-Parallel、Prompt-only Adaptive 及 Oracle-ASAP 的核心定量结果，包括严格成功率、CTS、wall-clock 和 Total Work；补充 BFCL、 τ^2 -Bench、DeepPlanning、Wide Search 与 Toolathlon 上的结果。只有在统一预算、多随机种子和置信区间均支持时，才写入“SOTA”或“超过闭源模型”的结论。]**在上述证据补齐前，本文的结论限于：MULTI-AGENT WORLD 将环境构建、编排任务合成、可验证两阶段学习与准确率-延迟联合评测连接为一个完整且可检验的研究框架。

AI USE STATEMENT

本中文工作稿使用生成式 AI 辅助梳理本地参考论文与实验记录、起草文字并检查 LaTeX 结构。作者需要逐条核对所有文献、公式、数据口径与实验主张，并对最终内容负责。代码、数据与实验结果不得仅依据模型生成内容宣称；最终投稿前应按照 ICLR 2027 AI Policy 补充完整披露。

ETHICS STATEMENT

当前环境由合成数据库和工具 fixture 构成，不应包含真实用户数据。公开数据与环境前仍需审计上游 schema、许可证、潜在隐私字段，以及形似 token/password 的合成值，避免误用或再分发风险。更强的任务编排能力也可能扩大自动化系统的操作范围；部署时应使用最小权限、隔离环境、可审计事件日志和高风险写操作确认机制。

REPRODUCIBILITY STATEMENT

最终版本将发布固定的数据清单与哈希、环境构造与验证脚本、完整任务级依赖/冲突标注、训练配置、Worker checkpoint 选择规则、所有基线控制器、事件 DAG 与 CTS 计算代码，以及逐随机种子的原始结果。附录已给出当前可核对的算法流程、动作与 Prompt 模板、原型超参数、资产哈希和结果报告协议；最终投稿将以冻结版本替换其中的精确占位，并补齐额外 Pareto 曲线与逐种子结果。

REFERENCES

- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment, 2025. URL <https://arxiv.org/abs/2506.07982>.
- Zihao Cheng, Hongru Wang, Zeming Liu, Xinyi Wang, Xiangrong Zhu, Yuhang Guo, Wei Lin, Jeff Z. Pan, and Yunhong Wang. Terminal-world: Scaling terminal-agent environments via agent skills, 2026. URL <https://arxiv.org/abs/2605.20876>.
- Guanting Dong, Junting Lu, Junjie Huang, Wanjuan Zhong, Longxiang Liu, Shijue Huang, Zhenyu Li, Yang Zhao, Xiaoshuai Song, Xiaoxi Li, et al. Agent-world: Scaling real-world environment synthesis for evolving general agent intelligence. *arXiv preprint arXiv:2604.18292*, 2026. URL <https://arxiv.org/abs/2604.18292>.
- Jeremy Hadfield, Barry Zhang, Kenneth Lien, Florian Scholz, Jeremy Fox, and Daniel Ford. How we built our multi-agent research system. Anthropic Engineering Blog, June 2025. URL <https://www.anthropic.com/engineering/multi-agent-research-system>.
- Kimi Team. Kimi K2.5: Visual agentic intelligence, 2026. URL <https://arxiv.org/abs/2602.02276>.
- Junlong Li, Wenshuo Zhao, Jian Zhao, Weihao Zeng, Haoze Wu, Xiaochen Wang, Rui Ge, Yuxuan Cao, Yuzhen Huang, Wei Liu, Junteng Liu, Zhaochen Su, Yiyang Guo, Fan Zhou, Lueyang Zhang, Juan Michelini, Xingyao Wang, Xiang Yue, Shuyan Zhou, Graham Neubig, and Junxian He. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution, 2025. URL <https://arxiv.org/abs/2510.25726>.

Table 6: 主要符号。

符号	含义	符号	含义
E	可重置工具环境	x, q_x	组合任务及其用户指令
u_j	第 j 个原子任务	G_x	隐藏任务依赖/冲突图
$\mathcal{D}_x$	有向数据或状态依赖边	$\mathcal{C}_x$	不可安全并发的冲突边
H_τ	rollout 实际产生的事件 DAG	K	最大 Worker 并发预算
B	CTS 硬预算	$W_{\max}$	总工作量硬预算
$C_K^*(x)$	并发预算 K 下的 Oracle span	$W^*(x)$	Oracle 总工具工作量
V_x^{final}	严格终局验证器	$V_{x,j}^{\text{proc}}$	仅供训练奖励使用的局部验证器

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=2GmDdhBdDk>.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=dHng200Jjr>.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=Yacmpz84TH>.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Yang Wu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning, 2026. URL <https://arxiv.org/abs/2602.10090>.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

A 复现细节与补充材料

本附录给出正文方法的可执行化约定、当前原型中已经固定的参数，以及正式主实验仍需补齐的配置与结果。除特别标注为“当前原型”的项目外，所有数据规模、预算系数和实验数值均应以最终冻结版本为准；尚未由代码、日志或数据快照确认的内容统一使用红色待补标记，不从设计目标反推实验事实。

A.1 符号、事件执行图与 CRITICAL TOOL STEPS

表 6 汇总正文及附录使用的主要符号。真实任务图 G_x 属于生成侧标注，执行图 H_τ 则由策略在一次 rollout 中实际产生，二者不能混用。

一般事件 DAG 定义。 令 $H_\tau = (\mathcal{V}_\tau, \mathcal{E}_\tau)$ ，每个节点 $v \in \mathcal{V}_\tau$ 表示一次 Orchestrator 编排动作、Orchestrator 直接工具动作或 Worker 工具动作；控制流、数据依赖、共享状态顺序以及 fork-join 关系构成边集 $\mathcal{E}_\tau$ 。V1 中每个实际尝试的动作权重均为 $w(v) = 1$ ，包括因参数错误或前置条件失败而返回错误观察的动作。模型内部推理 token、环境 observation、reward-side 验证器调用和尚未开始即被取消的分支权重为零；其代价分别通过 token、wall-clock 和取消率另行报告。

对 DAG 做拓扑排序，并递推

$$L(v) = w(v) + \max_{u \in \text{pred}(v)} L(u), \quad \max \emptyset = 0, \quad (22)$$

则一般异步执行图上的逻辑关键步数与总工作量分别为

$$\text{CTS}_{\text{DAG}}(\tau) = \max_{v \in \text{sink}(H_\tau)} L(v), \quad W(\tau) = \sum_{v \in \mathcal{V}_\tau} w(v). \quad (23)$$

式 22 只使用事件偏序，不依赖 Python 线程完成的偶然先后，因此可在不同硬件上复算。

同步实现与式 14 的关系。 当前方法采用 fork-join 批量委派：阶段 ℓ 的 Orchestrator 动作串行发生，随后集合 $\mathcal{I}_\ell$ 中的 Worker 分支并发运行，所有分支返回后才进入下一阶段。任一从该阶段入口到出口的路径都包含 $S_o^{(\ell)}$ 个主 Agent 动作，以及至多一个 Worker 分支；最长路径因此为

$$S_o^{(\ell)} + \max_{i \in \mathcal{I}_\ell} S_{w,i}^{(\ell)}. \quad (24)$$

不同阶段由 barrier 串接，最长路径可加，故式 22 恰好化为正式式 14。相反，把整个 episode 的全部 Worker 只取一次最大值会漏计多轮 fork-join，不能作为本文指标。

V1 计数约定。 一次 create_subagent、act、assign_task、批量 assign_tasks 或 finish 均计一个 Orchestrator 动作；一次批量派发的列表长度不增加 S_o ，但每个 Worker 的实际工具动作分别进入 W ，且最长分支进入 CTS。失败分支在超时前已经执行的动作照常计数；再次委派形成新的阶段并重新计数，因而局部重试不是免费的。V1 禁止 Worker 继续创建 Worker；若未来支持嵌套委派，则把 $S_{w,i}^{(\ell)}$ 替换为该子执行图递归计算的式 22。不同工具暂统一赋权 1；工具加权 CTS 只作为敏感性分析，并须使用训练集外固定的延迟权重。

A.2 WORLD BANK 与原子任务生成

每个 World Bank 条目至少包含工具 schema、可执行实现、规范初始 fixture、重置入口和来源记录。当前 bundle 的环境目录还保存 manifest、工具关系图与数据库文件；这些已有工具关系只能作为候选依赖的种子，不能替代式 5 所需的任务级读写集合与验证约束。

环境与原子任务 generate-verify-repair。 给定一个候选 MCP 描述，构建过程采用以下普通伪代码； $R_{\max}^{\text{env}}$ 与每个环境的采样任务数在冻结数据版本时确定。

1. 解析工具名称、参数、返回 schema 与应用实体；若缺少可重置结构化状态或无法离线执行，则丢弃候选。
2. 生成规范 fixture、工具实现与重置函数；对每个工具执行 schema、正常调用、非法参数、超时和重置测试。若失败，则在不超过 $R_{\max}^{\text{env}}$ 次内修复，否则丢弃环境。
3. 从已通过测试的工具和状态记录中采样自包含目标 q_j ，运行可执行 Oracle 得到 y_j^* 与终止状态，并生成 V_j^{atom} 。
4. 结合静态访问分析和调用前后状态差异，提出 Read_j 、 Write_j 、 Pre_j 与 Post_j ；若动态回放与标注不一致，则修复标注或删除任务。
5. 要求 Oracle 正例通过验证器；再依次删除必需字段、替换关键值、撤销目标状态变化并扰动无关字段。验证器必须拒绝前三类反例且不错误依赖无关字段。
6. 保存式 5 的原子记录以及生成 provenance；面向模型的数据导出时移除答案值、Oracle 程序和全部结构标注。

[待补：冻结 $R_{\max}^{\text{env}}$ 、每环境原子任务采样上限、环境/任务生成模型版本、解码参数及人工复核比例。]

A.3 组合任务生成与质量控制

结构约束组合伪代码。 输入为同一环境中的原子任务库、目标结构类型 $z \in \{\text{Serial}, \text{Parallel}, \text{Mixed}, \text{Contention}, \text{Recovery}\}$ 、节点数范围和并发预算 K ；输出为组合任务 x 或拒绝标记。

Table 7: 信息可见性。“奖励侧”表示仅环境宿主与评测脚本可读。

信息	Orchestrator	Worker	奖励侧
用户任务与允许工具 schema	✓	委派子问题与工具子集	✓
工具返回与 Worker 返回	✓	自身轨迹	✓
剩余 $B, K, W_{\max}$	✓	仅执行限制	✓
$G_x, \mathcal{D}_x, \mathcal{C}_x, \Gamma_j$	—	—	✓
Ground truth / Oracle 轨迹	—	—	✓
局部验证结果与过程分量	—	—	✓
严格终局判分	episode 结束后不反馈	—	✓

- 按 z 采样一个拓扑模板，并从原子库选择候选节点；用式 6 与式 7 构造候选 $\mathcal{D}_x$ 和 $\mathcal{C}_x$ 。
- 检查 DAG 无环、所有前置条件可满足，并验证 Parallel 节点无冲突、Contention 节点具有可复现冲突；若不满足模板，重新采样而不是修改标签迁就样本。
- 对依赖边绑定上游结果或状态事实，对冲突节点统一实体、版本或配额；Recovery 样本按固定种子为指定分支配置一次可恢复故障。
- 将各节点目标改写为一条连贯指令 q_x 。改写器不得出现节点 ID、边、“parallel”/“serial”、Oracle 顺序或局部标准答案。
- 编译局部验证器索引和式 8 的终局验证器，求解满足 $\mathcal{D}_x, \mathcal{C}_x$ 与 K 的 Oracle 调度，并记录 $W^*(x)$ 、 $C_K^*(x)$ 和 $P_K(x)$ 。
- 从规范初始状态回放 Oracle，要求终局验证通过；再运行遗漏节点、交换依赖顺序、并发冲突写与错误聚合等反例，要求其严格失败。
- 对自然语言、工具 schema 与导出 metadata 执行泄漏扫描；只有全部检查通过的实例才能进入任务池。

为避免生成器只满足形式模板，图级验证至少执行上游-下游两种顺序和允许时的并发顺序。只读候选的并发结果应与合法串行化一致；冲突候选则必须由事务、版本检查、锁或确定性错误返回产生可重复差异，而不能依赖线程调度时序。[待补：冻结五类任务的节点数、深度、宽度、冲突密度、Recovery 注入类型与最终 train/validation/test 数量。]

A.4 验证器、可见信息与无泄漏审计

表 7 固定三个运行角色的信息边界。局部验证器虽然在 rollout 期间可由 reward 服务执行，但其布尔结果、分数和失败原因都不能回写模型上下文；否则 Orchestrator 可以把验证器当作依赖图查询接口。

正式数据导出执行以下审计：(1) prompt 只包含输出字段及类型，不包含示例值或 ground truth；(2) 检查 prompt、工具描述和错误消息中是否出现答案值、节点 ID、依赖边序列化或 Oracle 工具序列；(3) 验证模型服务进程不能读取隐藏 metadata 文件；(4) 按环境 ID 去重，并检测跨 split 的任务 ID、规范化文本、工具组合和 DAG 同构类重叠；(5) 从测试 prompt 中随机抽样，人工核对自然语言是否暗示推荐并发策略。训练与评测日志还需保存 prompt 哈希和隐藏 metadata 哈希，以便复核实际输入与发布清单一致。

A.5 编排动作 SCHEMA 与 PROMPT

表 8 给出论文方法的最小动作接口。正文中的 finish 在当前代码中对应 submit_answer；字段语义一致。当前原型尚未暴露 act，且默认要求至少一次委派，这两项在正式主实验中必须关闭，才能学习“是否委派”。

Orchestrator system prompt 模板。 正式模板只说明角色、动作、预算和输出协议，不写入“优先并行”或“必须委派”等策略偏置：

你是任务编排器。根据用户任务、允许的工具、已返回的执行结果以及剩余预算，选择直接执行或创建并委派给使用冻结 checkpoint 的 Worker。你必须自行判断子任务边界与安全的执行顺序；不得假设分支独立，也不得编造工具结果。仅向 Worker 提

Table 8: Orchestrator 动作及其 JSON 字段。所有 arguments 必须是合法 JSON。

动作	必需字段	语义
act	tool_name:string, arguments:object	Orchestrator 直接调用一项任务工具
create_subagent	name:string, system_prompt:string, tool_names:list	注册使用冻结 Worker checkpoint 的角色与最小工具子集
assign_task	agent_name:string, prompt:string	同步执行单个委派; 同名 Worker 保留历史
assign_tasks	tasks:list[object]	并发执行一组委派并在全部返回后汇合
finish	answer:object	提交最终结构化答案并终止

供完成其局部目标所需的信息和工具。检查返回之间的缺失或冲突，并在预算内继续执行或重新委派。完成后使用 finish 提交与用户 schema 一致的严格 JSON。当前剩余预算为 $\langle B, K, W_{\max} \rangle$ 。

Worker system prompt 模板。

你是由 Orchestrator 调用的工具型 Worker。只完成委派目标 $\langle \text{SUBTASK} \rangle$ ，并只使用给定工具子集 $\langle \text{TOOLS} \rangle$ 。根据真实工具 observation 修正执行，不得猜测数据库或环境状态。返回一个简洁、可验证的 JSON，其中包含 Orchestrator 继续工作所需的事实；不要创建其他 Worker。

Prompt-only Adaptive 基线使用完全相同的动作 schema、system prompt、Worker checkpoint、工具、预算和采样参数，唯一差异是不对 Orchestrator 做第二阶段更新。Serial 与 Eager Fully-Parallel 基线只通过控制器限制可选拓扑，不获得隐藏 G_x ；Oracle-ASAP 才能读取 G_x ，且只作为诊断下界。**[待补：冻结并逐字附上最终英文 prompt、chat template 版本、tool-call parser 版本，以及所有基线的控制器伪代码。]**

A.6 训练配置与原型超参数

表 9 来自当前 Qwen3-8B Slime/GRPO 启动脚本，记录的是可复核的原型默认值，不是尚未完成的最终 Multi-Agent 主实验结果。Multi-Agent 路径目前只完成 CPU dry run；保存的 GPU provenance 只证明既有 Single-Agent 链路完成了 optimizer step 和权重同步。

正式主实验还必须固定：**[待补：Worker checkpoint 与选择轮次；训练/验证/测试条数；随机种子及数量； $\alpha, \beta, \xi, \gamma_p, \eta, \lambda, \mu$ ； $W_{\max}$ 的任务归一化规则；budget sampling 分布；总 optimizer steps、warmup、训练 wall-clock、checkpoint 选择与 early stopping；最终硬件型号和软件 commit。]** 主结果按照正文扫描 $K \in \{1, 2, 4, 8\}$ 与 $B/C_K^*(x) \in \{1.0, 1.5, 2.0, 3.0\}$ ；每个运行点必须使用同一冻结 Worker 和配对随机种子。

A.7 BENCHMARK 与评测协议映射

外部 benchmark 的作用与 MULTI-AGENT WORLD 组合任务不同：前者主要检验底层工具执行和分布外迁移，后者才直接支持隐藏拓扑下的严格成功率-CTS 结论。表 10 预先规定其论文角色，避免把不同官方指标直接求平均。

所有方法共享模型、Worker checkpoint、工具、初始状态、上下文、temperature、top-p、单 Worker 最大步数、 K 、 B 与 $W_{\max}$ 。预算越界样本计为失败，不从分母中删除。每个任务采用配对种子；严格成功率报告 bootstrap 95% 置信区间，方法间二元成功差异使用配对检验。CTS 与 wall-clock 同时报告，并按工具类型和 K 给出 Spearman/Pearson 相关性，避免把逻辑步数解释为真实秒数。

A.8 完整结果与敏感性分析模板

[待补：完整主表：补入预注册公共预算点下各方法的均值、95% 置信区间与配对检验，并同时给出对应模型调用、token 和峰值并发。]

Table 9: 当前原型中已固定的训练与 rollout 默认值。

配置	当前值
基础 actor	Qwen3-8B
优化算法	GRPO; 每 prompt 采样 $G = 8$
epoch / rollout batch / global batch	3 / 16 prompts / 128 trajectories
每 rollout 优化步	1
最大 response / context	1,024 / 28,672 tokens
rollout temperature / top- p	1.0 / 1.0
optimizer / learning rate	Adam / 1×10^{-6} , constant decay
weight decay / (β_1, β_2) / grad clip	0.017 / (0.9, 0.999) / 1.0
GRPO clip low / high	0.2 / 0.4
KL coefficient / entropy coefficient	0.0 / 0.0
actor 结构	8 GPU colocate, TP= 2, sequence parallel
rollout 服务	4 个可更新 actor replicas + 4 个冻结 Worker replicas
SGLang static memory fraction	0.65
Orchestrator 最大 turn	16
最大已注册 Sub-agent	8 (不是已实现的 K 硬约束)
Worker 最大迭代 / context / 单次生成	16 / 28,672 / 1,024 tokens
observation / Worker 结果 / 委派结果上限	3,000 / 6,000 / 16,000 characters
actor / Worker thinking	均关闭
当前原型强制委派 / format bonus	true / 0.0

Table 10: Benchmark、能力维度与报告方式。

数据集	主要能力与用途	本文报告方式
MULTI-AGENT WORLD held-out	未知依赖、共享状态、恢复与拓扑选择	严格成功率-CTS 曲线、 W 、wall-clock、token; 按五类结构与 OOD 切片
BFCL (Patil et al., 2025)	函数选择、参数生成和工具调用协议; Worker 能力保持性	官方 accuracy; 作为 Executor 指标, 不替代编排主指标
τ^2 -Bench (Barres et al., 2025)	有状态、多轮工具交互与约束满足	官方严格任务指标; [待补: 冻结版本、domain 与重复次数]
DeepPlanning	长程规划与多步约束; 旧记录仅用于 Worker checkpoint 诊断	官方 avg_acc; [待补: 补充版本、任务子集与评测脚本 commit]
Wide Search	可并行宽搜索与延迟; 用于外部编排泛化	严格成功率、CTS、 W 与 wall-clock; [待补: 补充正式数据集名称和版本]
Toolathlon (Li et al., 2025)	跨工具、长程组合执行	官方任务成功指标; 另报动作成本, [待补: 冻结 split 和 evaluator 版本]
Shopping / Travel	旧实验中的 Executor 诊断切片	分别保留严格成功率与 case_acc; 不并入主 Pareto 平均

除表 11 外, 最终附录逐一报告: (1) 每个 K 与 B/C_K^* 的 Accuracy-Budget AUC; (2) 未见环境、未见工具组合、未见 DAG 形状和更长任务; (3) 固定成功率所需的最小 CTS; (4) token/FLOPs、峰值并发、handoff、失败重派、冲突和重复工作; (5) 按工具类型校准的 wall-clock p50/p90。不得用字段级准确率替代严格成功率。

消融。 统一在同一任务和配对种子上比较: 去掉 R_{proc} ; 去掉 CTS 惩罚; 去掉 W 惩罚; 不做 budget conditioning; 不平衡结构采样; 允许 Worker 联合更新; 移除失败重做; 以及只用 Serial 或 Parallel 数据训练。[待补: 补入各消融的 Pareto AUC、固定预算严格成功率及相对完整方法的配对置信区间。]

[待补：补入总体及 Serial/Parallel/Mixed/Contention/Recovery 五个结构桶的 Strict Success-CTS Budget 曲线；包含 95% CI、Oracle-ASAP 和各方法非支配点。]

Figure 4: 完整准确率-逻辑延迟 Pareto 曲线占位。

Table 11: 固定预算下的完整主结果模板。每个单元格最终报告均值及 95% CI。

方法	Strict Succ.	CTS	W	Wall-clock
Single-Agent ReAct	—	—	—	—
Serial Multi-Agent	—	—	—	—
Eager Fully-Parallel	—	—	—	—
Prompt-only Adaptive	—	—	—	—
Learned Adaptive	—	—	—	—
Oracle-ASAP	—	—	—	—

A.9 案例、事件日志与失败分类

每个案例发布用户指令、可见工具、隐藏 G_x (仅在事后分析中展示)、所有方法的事件 DAG、状态版本、局部/终局验证结果、CTS、W 和 wall-clock。案例选择在查看模型结果前按结构随机抽取，以避免只展示成功轨迹。

Table 12: 事件日志的最小字段。

字段	含义
episode/event/parent	episode ID、事件 ID 与 DAG 父事件
actor/action/arguments	Orchestrator 或 Worker、动作类型及规范化参数哈希
dispatch/start/end	逻辑派发序号与单调 wall-clock 时间戳
state_version	动作前后共享状态版本与受影响对象
status/error	success、invalid、timeout、conflict、cancelled 等结果
verifier_ids	reward-side 触发的局部/终局验证器索引；不写入模型 observation
token/tool counts	主/副模型 token 与实际尝试的工具动作数

Table 13: 失败类型及可复核判据。

类型	日志判据
Decomposition	必需子目标缺失、重复或边界使局部目标不可独立验证
Dependency	下游在所需输出/状态事实产生前被派发
Contention	无合法串行化的并发读写、版本或配额冲突
Handoff	上游已产生必要事实，但委派 prompt 未传递或错误转换
Aggregation	局部分支正确，最终答案遗漏、冲突或计算错误
Recovery	忽略错误/不完整返回，或对失败分支作出错误重派决策
Budget	B 或 $W_{\max}$ 耗尽，或动作试图超过并发上限 K
Executor	拆分与顺序合理，但 Worker 工具选择、参数或局部恢复失败

[待补：分别补入至少一个 Serial、Parallel、Mixed、Contention 与 Recovery 案例；并由两名标注者独立复核失败分类，报告一致率及分歧裁决规则。]

A.10 数据、代码与实验声明

表 14 记录当前工作目录可复核的资产快照。v2 与 v3 有 9,005 个任务 ID 重叠，故任务并集为 13,946，不能直接相加；两个版本引用的环境并集为 600，当前 archive 含 6,330 个工具

schema。v3 的 10,860 条记录只有聚合路径统计，尚不等于通过第 A.3 节检查的完整组合任务。

Table 14: 当前资产快照及 SHA-256；最终投稿须以冻结 release 重新生成。

资产	条数	SHA-256
v3 verified snapshot	10,860	3a177bea40f1b35ca5760f948f9bf3cb 3b95ceed05687ef4424641067720b3f0
v3 Slime JSONL	10,860	283e31d9b4dd2c9c6787fa42c6b0fb89 4048f95fb7b83ddf947d41284d231f75
v2 verified snapshot	12,091	1d7de513bac79b44c8bd0e3e946a2e61 3b154ddf4c96e38c59f2e0b1018f95f3
v2 Slime JSONL	12,091	42b549974b1e2aff9210fb4991802962 b794c0a8e4b7be4bfe4243ef567c5844
工具环境 aggregate	600 envs	1bf64660633c8949dd6cc3a0d32bfade 5caf7a04fbc9671c8737469d74a71127

当前保存的训练环境记录包括 Slime commit fb42ae456fac8166afb604f13b30d22bb3c75053c、Python 3.12.3、PyTorch 2.9.1+cu128、CUDA 12.8、SGLang 0.5.10.post1 和 Ray 2.54.0。现有 provenance 支持 Single-Agent GPU 链路和 Multi-Agent CPU dry run，不支持声称完整 Multi-Agent GPU optimizer step、CTS 结果或 Pareto 优势。

最终 release 应包含：固定的数据 manifest 与逐文件哈希；环境/任务生成、验证与泄漏扫描脚本；任务级 Γ_j 、 G_x 和 Oracle 标注的受控评测副本；全部 prompt、动作 schema、基线控制器与预算终止规则；Worker checkpoint 选择日志；事件 DAG 和 CTS 复算工具；逐任务逐种子的原始输出及统计脚本。公开前还需审计上游 MCP/schema 的许可证、合成 fixture 中形似凭证或个人信息的字段，以及模型和 benchmark 的再分发条款。**[待补：补入最终仓库 commit、数据 release DOI/URL、许可证清单、硬件型号、训练日志 manifest 与公开日期。]**